

Week 4 Report

Docker & Docker Compose
DevOps Internship | Davine Technologies

1. What is Docker?

Docker is a platform that packages an application and its dependencies into a lightweight, portable unit called a **container**. Containers share the host operating system kernel, so they start in seconds and use far fewer resources than virtual machines. The same container runs identically on a laptop, a test server or the cloud, which removes the 'it works on my machine' problem.

2. Docker Architecture

Docker uses a client-server architecture. The **Docker client** (the docker command) sends requests to the **Docker daemon** (dockerd), which builds images, runs containers and manages networks and volumes.

Images are stored in a **registry** such as Docker Hub.

```
docker CLI --> Docker Daemon --> Images / Containers / Volumes / Networks
|
+--> Registry (Docker Hub): pull / push
```

3. Images vs Containers

	Image	Container
Definition	Read-only template with the app, runtime and libraries	Running instance of an image
Created with	docker build	docker run
State	Immutable layers	Adds a writable layer; can be started, stopped, removed
Analogy	Class / recipe	Object / cooked dish

4. Dockerfile

A **Dockerfile** is a text file of instructions for building an image. Each instruction creates a cached layer, so copying dependency files before the source code speeds up rebuilds.

```
FROM node:20-alpine          # base image
WORKDIR /app                  # working directory
COPY package*.json ./
RUN npm install --omit=dev    # install dependencies
COPY . .                      # copy source
ENV PORT=3000                 # environment variable
EXPOSE 3000                   # document the port
USER node                     # run as non-root
CMD ["node", "server.js"]     # start command
```

5. Docker Volumes

Data inside a container is lost when the container is removed. **Volumes** are managed by Docker and store data outside the container's lifecycle, which is essential for databases. A **bind mount** maps a host folder into the container instead.

```
docker volume create app-data
docker run -d -v app-data:/data myapp
docker volume ls
```

6. Docker Networks

Docker networks let containers communicate. On a user-defined **bridge network**, containers reach each other by name through built-in DNS (for example the backend connects to the host 'db'). Networks also isolate groups of containers from each other.

Driver	Use
bridge	Default single-host network; custom bridges give name resolution
host	Container shares the host network stack
none	No networking

```
docker network create app-net
docker run -d --network app-net --name db postgres:16-alpine
```

7. Environment Variables and Port Mapping

Environment variables (-e or ENV) configure applications without changing the image, such as database credentials. **Port mapping** (-p HOST:CONTAINER) publishes a container port on the host so it can be reached from a browser, for example -p 8080:80.

8. Docker Compose

Docker Compose defines and runs multi-container applications with one YAML file. It declares services, networks, volumes, environment variables and start-up order, and brings everything up with a single command.

```
services:
  frontend: { build: ./frontend, ports: ["8080:80"] }
  backend:  { build: ./backend, depends_on: [db] }
  db:       { image: postgres:16-alpine, volumes: [db-data:/var/lib/postgresql/data] }
volumes: { db-data: {} }
networks: { default: { name: app-net } }
```

```
docker compose up -d --build
docker compose down
```

9. Docker Hub

Docker Hub is the public registry for sharing images. Official images (nginx, node, postgres) can be pulled directly, and your own images are published by tagging them with your username and pushing.

```
docker login
docker tag docker-week4-app:1.0 <username>/docker-week4-app:1.0
docker push <username>/docker-week4-app:1.0
docker pull <username>/docker-week4-app:1.0
```

10. Docker Best Practices

Practice	Why
Use small base images (alpine)	Faster builds and pulls, smaller attack surface
Add a .dockerignore	Keeps node_modules, .git and secrets out of the image
Copy dependency files before source	Uses layer caching to speed up rebuilds
Run as a non-root user	Limits damage if the container is compromised
Never bake secrets into images	Use environment variables or secret managers
Pin image versions	Reproducible builds; avoid the 'latest' tag
One process per container	Easier scaling and troubleshooting

11. Hands-on Activity Summary

Containerized a web application as three services: an Nginx frontend, a Node.js backend and a PostgreSQL database. Built custom images, pushed them to Docker Hub and deployed the stack with Docker Compose. A named volume persisted database data, a custom bridge network connected the containers by service name, and the health and visits endpoints confirmed everything communicated successfully.

12. Conclusion

Week 4 introduced containerization, the foundation for CI/CD pipelines with Jenkins and for orchestration with Kubernetes in later weeks.